

CSC 4520/6520

Design & Analysis of Algorithms

CHAPTER 2: ASYMPTOTIC ANALYSIS OF ALGORITHMS

ABDULLAH BAL, PHD

Objectives



Recap

Analysis of Algorithms
Sequential Search, Euclid's, Sieve
The best, worst, and average case
Order of Growth Comparison



Asymptotic Order of Growth



L'Hopital, Stirling



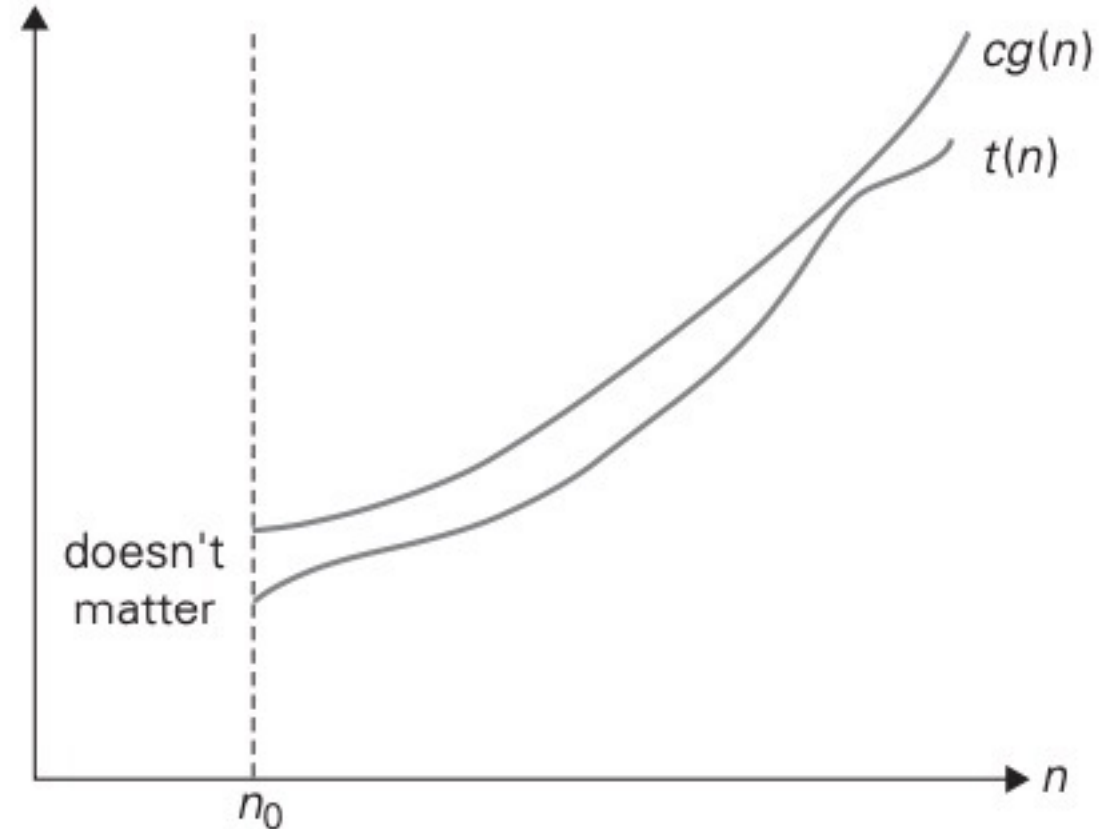
Nonrecursive Algorithms

Asymptotic Notations

- The efficiency analysis framework concentrates on the order of growth of an algorithm's **basic operation** count as the principal indicator of the algorithm's efficiency.
- To **compare** and **rank** such orders of growth, computer scientists use three notations:
 - O (big oh)
 - Θ (big theta)
 - Ω (big omega)

Big-oh

- A function $t(n)$ is said to be in $O(g(n))$, denoted $t(n) \in O(g(n))$,
- If $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that
- $t(n) \leq cg(n)$ for all $n \geq n_0$.



Asymptotic order of growth (O)

- $O(g(n))$ is the set of all functions with a **lower or same order of growth** as $g(n)$ (to within a constant multiple, as n goes to infinity).
- In other words, $g(n)$ **must be faster** than $t(n)$ in terms of order of growth.
- All the following assertions are true.

$$n \in O(n^2), \quad 100n + 5 \in O(n^2), \quad \frac{1}{2}n(n-1) \in O(n^2).$$

$$n^3 \notin O(n^2), \quad 0.00001n^3 \notin O(n^2), \quad n^4 + n + 1 \notin O(n^2).$$

Big-oh

$$t(n) \leq cg(n) \text{ for all } n \geq n_0.$$

- Let us **formally prove** following assertion:

$$100n + 5 \in O(n^2).$$

- Indeed,

$$100n + 5 \leq 100n + n \text{ (for all } n \geq 5) = 101n \leq 101n^2.$$

- Thus, as values of the constants c and n_0 required by the definition, we can take 101 and 5, respectively.

Big-oh

- Note that the definition **gives us a lot of freedom** in choosing specific values for constants c and n_0 .

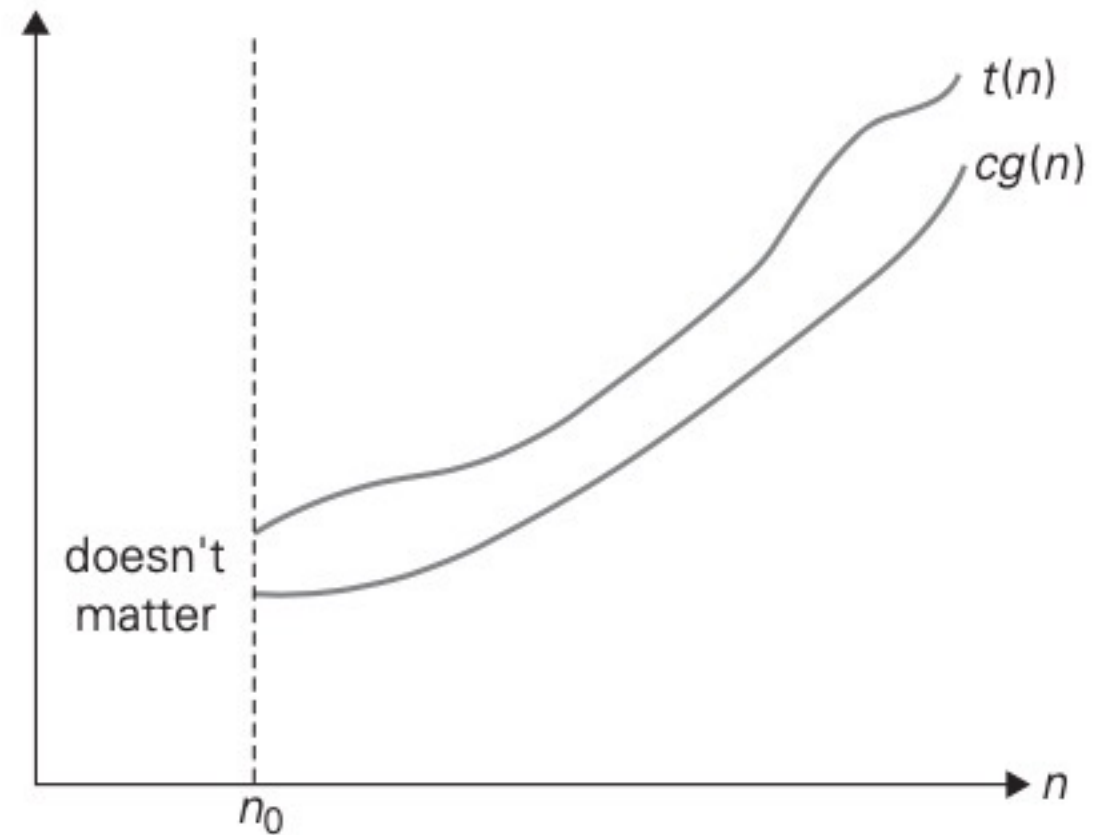
- For example, we could also reason that

$$100n + 5 \leq 100n + 5n \text{ (for all } n \geq 1) = 105n$$

to complete the proof with $c = 105$ and $n_0 = 1$.

Big-omega

- A function $t(n)$ is said to be in $\Omega(g(n))$, denoted $t(n) \in \Omega(g(n))$,
- If $t(n)$ is bounded below by some positive constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that
- $t(n) \geq cg(n)$ for all $n \geq n_0$.



Asymptotic order of growth (Ω)

- The second notation, $\Omega(g(n))$, stands for the set of all functions with **a higher or same order of growth** as $g(n)$ (to within a constant multiple, as n goes to infinity).
- In other words, $g(n)$ **must be slower** than $t(n)$ in terms of order of growth.
- For example:

$$n^3 \in \Omega(n^2), \quad \frac{1}{2}n(n-1) \in \Omega(n^2), \quad \text{but } 100n + 5 \notin \Omega(n^2).$$

Big-omega

- Here is an example of the formal proof that

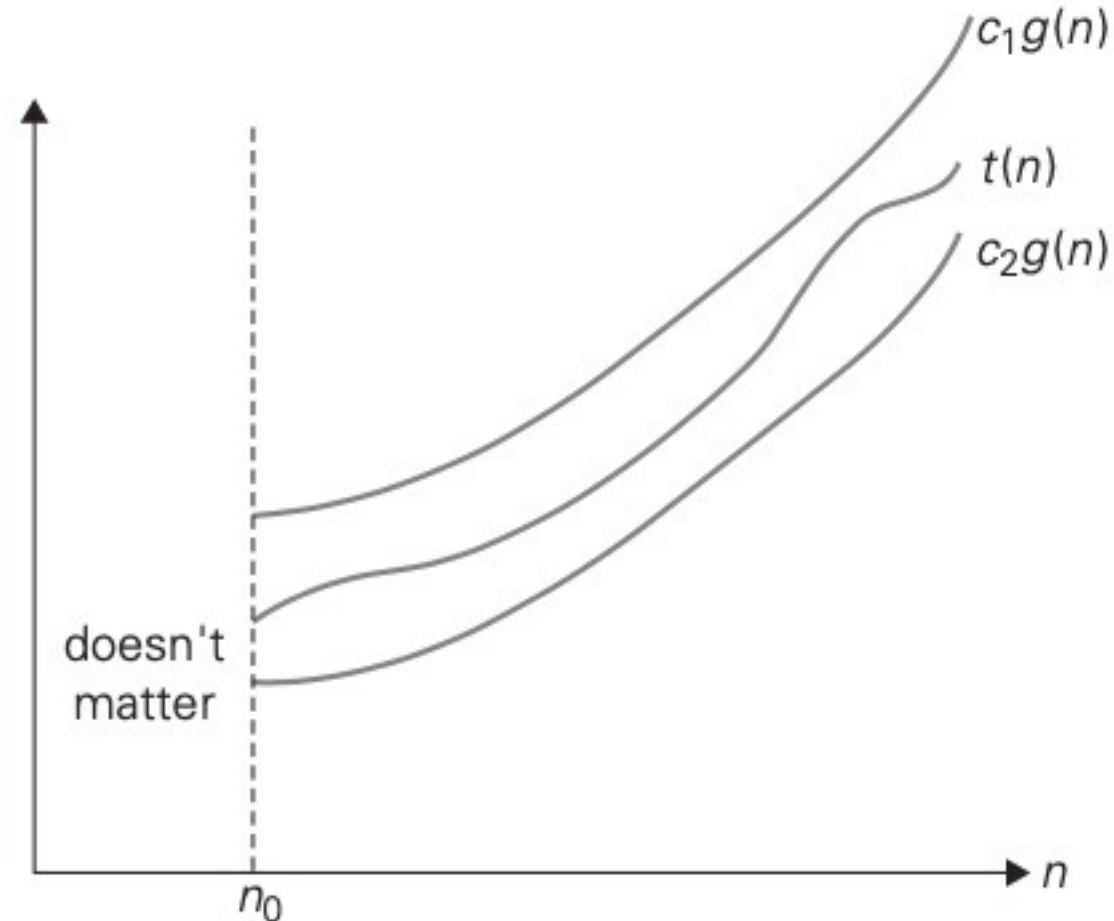
$$n^3 \in \Omega(n^2)$$

$$n^3 \geq n^2 \text{ for all } n \geq 0,$$

- i.e., we can select $c=1$ and $n_0=0$

Big-theta

- A function $t(n)$ is said to be in $\Theta(g(n))$, denoted $t(n) \in \Theta(g(n))$,
- If $t(n)$ is bounded both above and below by some positive constant multiples of $g(n)$ for all large n , i.e., if there exist some positive constants c_1 and c_2 and some nonnegative integer n_0 such that
- $c_2g(n) \leq t(n) \leq c_1g(n)$ for all $n \geq n_0$.



Asymptotic order of growth (Θ)

- $\Theta(g(n))$ is the set of all functions that have the same order of growth as $g(n)$ (to within a constant multiple, as n goes to infinity).
- Thus, every quadratic function $an^2 + bn + c$ with $a > 0$ is in $\Theta(n^2)$.

Big-theta

For example, let us prove that $\frac{1}{2}n(n-1) \in \Theta(n^2)$. First, we prove the right inequality (the upper bound):

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2 \quad \text{for all } n \geq 0.$$

Second, we prove the left inequality (the lower bound):

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{2}n^2 - \frac{1}{2}n \frac{1}{2}n \quad (\text{for all } n \geq 2) = \frac{1}{4}n^2.$$

Hence, we can select $c_2 = \frac{1}{4}$, $c_1 = \frac{1}{2}$, and $n_0 = 2$.

Establishing order of growth using the definition

Definition: $f(n)$ is in $O(g(n))$ if order of growth of $f(n) \leq$ order of growth of $g(n)$ (within constant multiple),
i.e., there exist positive constant c and non-negative integer n_0 such that

$$f(n) \leq cg(n) \text{ for every } n \geq n_0$$

Example 1: $10n$ is $O(n^2)$

Solution: $10n \leq n^2$ for $n \geq 10$

$c=1$ and $n_0=10$

Example 2: $5n+20$ is $O(n)$

Solution: $5n+20 \leq 10n$ for $n \geq 4$

$c=10$ and $n_0=4$

Establishing order of growth using limits

- A much more convenient method for comparing the orders of growth of two specific functions is based on computing the limit of the ratio of two functions in question.
- Note that the first two cases mean that $t(n) \in O(g(n))$, the last two mean that $t(n) \in \Omega(g(n))$, and the second case means that $t(n) \in \Theta(g(n))$.

$$\lim_{n \rightarrow \infty} t(n)/g(n) = \begin{cases} 0 & \text{order of growth of } T(n) < \text{order of growth of } g(n) & \longrightarrow & O(g(n)) \\ c > 0 & \text{order of growth of } T(n) = \text{order of growth of } g(n) & \longrightarrow & \Theta(g(n)) \\ \infty & \text{order of growth of } T(n) > \text{order of growth of } g(n) & \longrightarrow & \Omega(g(n)) \end{cases}$$

Examples: 1) $10n$ vs. n^2
2) $n(n+1)/2$ vs. n^2

L'Hôpital's rule and Stirling's formula

- The limit-based approach is often more convenient.
- It can take advantage of the powerful calculus techniques developed for computing limits, such as L'Hôpital's rule

L'Hôpital's rule: If $\lim_{n \rightarrow \infty} f(n) = \lim_{n \rightarrow \infty} g(n) = \infty$ and the derivatives f' , and g' exist, then

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$$

L'Hôpital's rule and Stirling's formula

Stirling's formula: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$ for large values of n .

Examples: 1) $\log n$ vs. n

2) $n!$ vs. 2^n

L'Hôpital's rule and Stirling's Examples

EXAMPLE 1 Compare the orders of growth of $\frac{1}{2}n(n-1)$ and n^2 . (This is one of the examples we used at the beginning of this section to illustrate the definitions.)

$$\lim_{n \rightarrow \infty} \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n^2 - n}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2}.$$

Since the limit is equal to a positive constant, the functions have the same order of growth or, symbolically, $\frac{1}{2}n(n-1) \in \Theta(n^2)$. ■

L'Hôpital's rule and Stirling's Examples (Cont.)

EXAMPLE 2 Compare the orders of growth of $\log_2 n$ and $\sqrt{n}$. (Unlike Example 1, the answer here is not immediately obvious.)

$$\lim_{n \rightarrow \infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{(\log_2 n)'}{(\sqrt{n})'} = \lim_{n \rightarrow \infty} \frac{(\log_2 e) \frac{1}{n}}{\frac{1}{2\sqrt{n}}} = 2 \log_2 e \lim_{n \rightarrow \infty} \frac{1}{\sqrt{n}} = 0.$$

Since the limit is equal to zero, $\log_2 n$ has a smaller order of growth than $\sqrt{n}$.

L'Hôpital's rule and Stirling's Examples (Cont.)

EXAMPLE 3 Compare the orders of growth of $n!$ and 2^n . (We discussed this informally in Section 2.1.) Taking advantage of Stirling's formula, we get

$$\lim_{n \rightarrow \infty} \frac{n!}{2^n} = \lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n}{2^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \frac{n^n}{2^n e^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \left(\frac{n}{2e}\right)^n = \infty.$$

Thus, though 2^n grows very fast, $n!$ grows still faster. We can write symbolically that

$$n! \in \Omega(2^n)$$

Orders of growth of some important functions

- All **logarithmic functions** $\log_a n$ belong to the same class:

$\Theta(\log n)$ no matter what the logarithm's base $a > 1$ is

- All **polynomials of the same degree** k belong to the same class:

$$a_k n^k + a_{k-1} n^{k-1} + \dots + a_0 \in \Theta(n^k)$$

- **Exponential functions** a^n have different orders of growth for different a 's

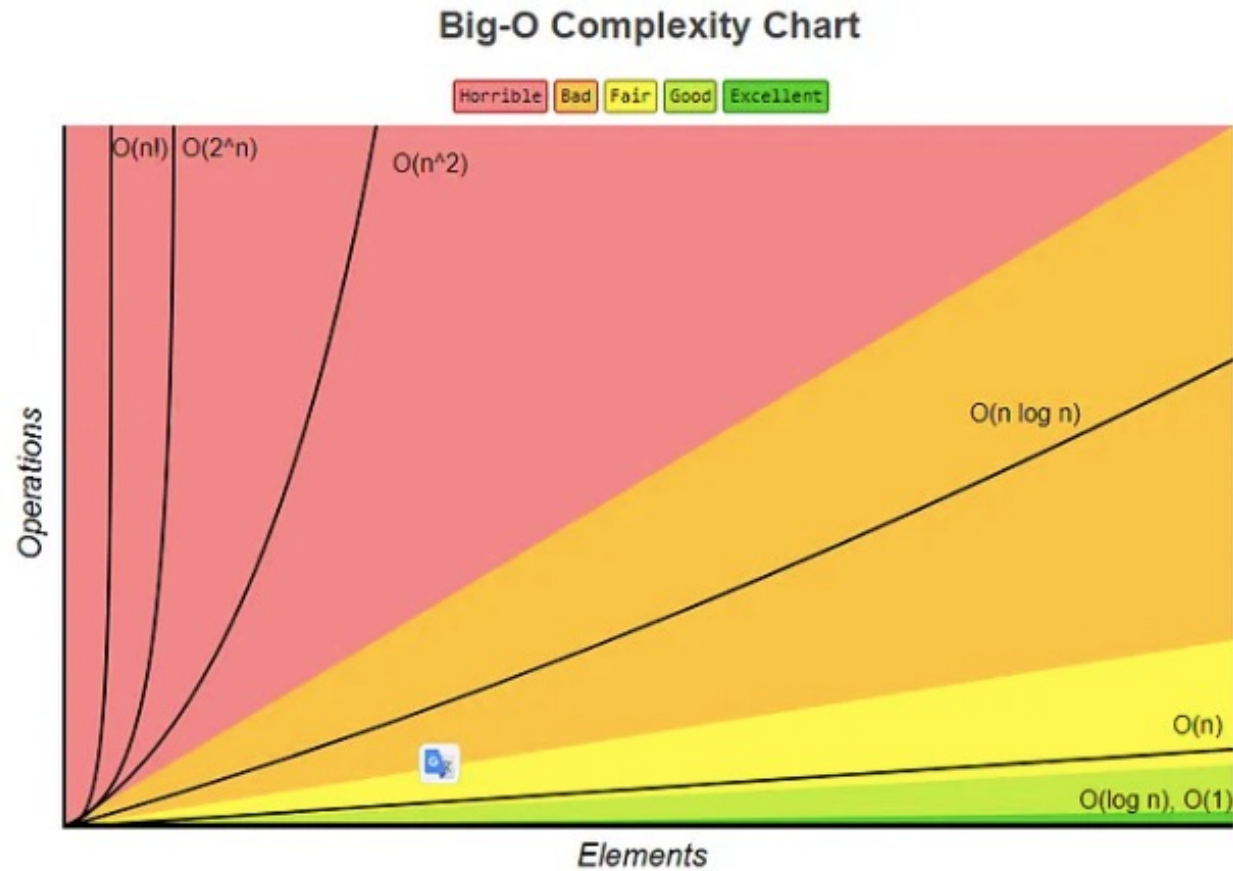
- Order $\log n < \text{order } n^\alpha \ (\alpha > 0) < \text{order } a^n < \text{order } n! < \text{order } n^n$

Basic asymptotic efficiency classes

- There are still **infinitely many such classes**.
- For example, **the exponential functions** a^n **have different orders of growth** for different values of base a .
- Therefore, it may come as a surprise that the time efficiencies of a large number of algorithms fall into only a few classes.

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n-log-n or linearithmic
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

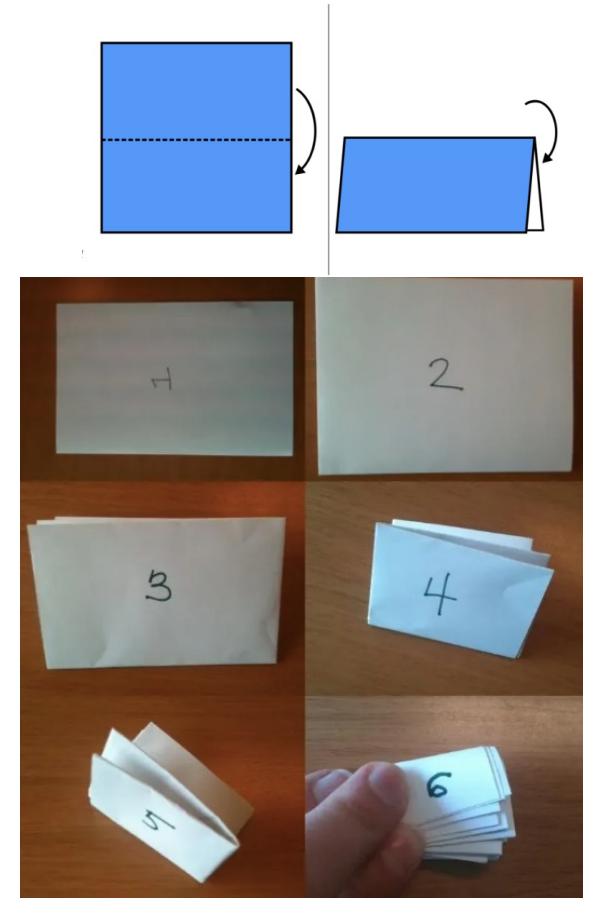
Basic asymptotic efficiency classes



Example: The Power of Exponential Growth

1. Take a piece of paper and fold it in half. How many layers is it? **(2)**
2. Fold it in half again. How many layers is it now? **(4)**
3. Fold it in half again. How many layers now? **(8)**
4. What do you expect to happen if you fold it in half a fourth time? **(16 layers)**
5. Can you identify the pattern? (if n is the number of times you folded, the pattern is 2^n , or $2 \times 2 \times 2 \times 2 \times \dots n$ times)

If you fold it in half 42 times, what will be its thickness?



Example: The Power of Exponential Growth

- Thickness of a paper = 0.1 mm
- $2^{42} \times 0.1 \text{ mm} = 439,804 \text{ kilometers}$
- The average distance from the Earth to the Moon is about 384,400 km

Time efficiency of NONRECURSIVE algorithms

Time efficiency of nonrecursive algorithms

General Plan for Analysis

- Decide on parameter n indicating input size
- Identify algorithm's basic operation
- Determine worst, average, and best cases for input of size n
- Set up a sum for the number of times the basic operation is executed
- Simplify the sum using standard formulas and rules

Useful summation formulas and rules

- In particular, we use especially frequently two basic rules of sum manipulation

$$\sum_{i=l}^u ca_i = c \sum_{i=l}^u a_i,$$
$$\sum_{i=l}^u (a_i \pm b_i) = \sum_{i=l}^u a_i \pm \sum_{i=l}^u b_i,$$

- Two summation formulas

$$\sum_{i=l}^u 1 = u - l + 1 \quad \text{where } l \leq u \text{ are some lower and upper integer limits,}$$

$$\sum_{i=0}^n i = \sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2).$$

Example 1: Maximum element

EXAMPLE 1 Consider the problem of finding the value of the largest element in a list of n numbers. For simplicity, we assume that the list is implemented as an array. The following is pseudocode of a standard algorithm for solving the problem.

ALGORITHM *MaxElement*($A[0..n - 1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0..n - 1]$ of real numbers

//Output: The value of the largest element in A

$maxval \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $A[i] > maxval$

$maxval \leftarrow A[i]$

return $maxval$

Example 1: Maximum element (Cont.)

- The obvious measure of an input's size here is the number of elements in the array, i.e., n .
- The operations that are going to be executed most often are in the algorithm's for loop.
- There are two operations in the loop's body: the comparison $A[i] > \text{maxval}$ and the assignment $\text{maxval} \leftarrow A[i]$.
- Which of these two operations should we consider basic?
- Since the comparison is executed on each repetition of the loop and the assignment is not, we should consider the comparison to be the algorithm's basic operation.
- Note that the number of comparisons will be the same for all arrays of size n ; therefore, in terms of this metric, there is no need to distinguish among the worst, average, and best cases here.

Example 1: Maximum element (Cont.)

- Let us denote $C(n)$ the number of times this comparison is executed and try to find a formula expressing it as a function of size n .
- The algorithm makes one comparison on each execution of the loop, which is repeated for each value of the loop's variable i within the bounds 1 and $n - 1$, inclusive.
- Therefore, we get the following sum for $C(n)$:

$$C(n) = \sum_{i=1}^{n-1} 1.$$

- This is an easy sum to compute because it is nothing other than 1 repeated $n - 1$ times. Thus,

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n).$$

Example 2: Element uniqueness problem

EXAMPLE 2 Consider the *element uniqueness problem*: check whether all the elements in a given array of n elements are distinct. This problem can be solved by the following straightforward algorithm.

ALGORITHM *UniqueElements*($A[0..n - 1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0..n - 1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[i] = A[j]$ **return false**

return true

Example 2: Element uniqueness problem (Cont.)

- The natural measure of the **input's size here is again n** , the number of elements in the array.
- Since the **innermost loop** contains a single operation (the comparison of two elements), we should consider it as the algorithm's **basic operation**.
- Note, however, that the number of element comparisons depends not only on n but also on whether there are equal elements in the array and, if there are, which array positions they occupy.
- We will limit our investigation to the **worst case** only.
- By definition, the worst case input is an array for which the number of element comparisons $C_{worst}(n)$ is the largest among all arrays of size n .

Example 2: Element uniqueness problem (Cont.)

- There are two kinds of **worst-case** inputs for which the algorithm does not exit the loop prematurely:
 - 1) Arrays with no equal elements
 - 2) Arrays in which the last two elements are the only pair of equal elements.
- **One comparison** is made for each repetition of the innermost loop, i.e., for each value of the loop variable j between its limits $i + 1$ and $n - 1$;
- This is **repeated for each value of the outer loop**, i.e., for each value of the loop variable i between its limits 0 and $n - 2$.

Example 2: Element uniqueness problem (Cont.)

```
for  $i \leftarrow 0$  to  $n - 2$  do  
    for  $j \leftarrow i + 1$  to  $n - 1$  do  
        if  $A[i] = A[j]$  return false  
return true
```

$$\begin{aligned} C_{\text{worst}}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) \\ &= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \\ &= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2). \end{aligned}$$

Example 3: Matrix multiplication

EXAMPLE 3 Given two $n \times n$ matrices A and B , find the time efficiency of the definition-based algorithm for computing their product $C = AB$. By definition, C is an $n \times n$ matrix whose elements are computed as the scalar (dot) products of the rows of matrix A and the columns of matrix B :

$$\begin{array}{c} \text{row } i \end{array} \begin{array}{c} A \\ \left[\begin{array}{|c|c|c|c|c|} \hline \square & \square & \square & \square & \square \\ \hline \end{array} \right] \end{array} * \begin{array}{c} B \\ \left[\begin{array}{|c|} \hline \square \\ \square \\ \square \\ \square \\ \square \\ \hline \end{array} \right] \end{array} \begin{array}{c} \text{col. } j \end{array} = \begin{array}{c} C \\ \left[\begin{array}{|c|} \hline C[i,j] \\ \hline \end{array} \right] \end{array}$$

where $C[i, j] = A[i, 0]B[0, j] + \cdots + A[i, k]B[k, j] + \cdots + A[i, n - 1]B[n - 1, j]$
for every pair of indices $0 \leq i, j \leq n - 1$.

Example 3: Matrix multiplication (Cont.)

ALGORITHM *MatrixMultiplication*($A[0..n-1, 0..n-1]$, $B[0..n-1, 0..n-1]$)
//Multiplies two square matrices of order n by the definition-based algorithm
//Input: Two $n \times n$ matrices A and B
//Output: Matrix $C = AB$
for $i \leftarrow 0$ **to** $n - 1$ **do**
 for $j \leftarrow 0$ **to** $n - 1$ **do**
 $C[i, j] \leftarrow 0.0$
 for $k \leftarrow 0$ **to** $n - 1$ **do**
 $C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$
return C

Example 3: Matrix multiplication (Cont.)

- We consider multiplication as the basic operation.
- Let us set up a sum for the total number of multiplications $M(n)$ executed by the algorithm.
- Since this count depends only on the size of the input matrices, we do not have to investigate the worst-case, average-case, and best-case efficiencies separately.
- Obviously, there is just one multiplication executed on each repetition of the algorithm's innermost loop, which is governed by the variable k ranging from the lower bound 0 to the upper bound $n - 1$.

Example 3: Matrix multiplication (Cont.)

- Therefore, the number of multiplications made for every pair of specific values of variables i and j is

$$\sum_{k=0}^{n-1} 1,$$

- The total number of multiplications $M(n)$ is expressed by the following triple sum:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1.$$

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3.$$

Example 3: Matrix multiplication (Cont.)

- If we now want to estimate the running time of the algorithm on a particular machine, we can do it by the product

$$T(n) \approx c_m M(n) = c_m n^3,$$

where c_m is the time of one multiplication on the machine in question.

- We would get a more accurate estimate if we took into account the time spent on the additions, too:

$$T(n) \approx c_m M(n) + c_a A(n) = c_m n^3 + c_a n^3 = (c_m + c_a) n^3,$$

where c_a is the time of one addition.

- Note that the estimates differ only by their multiplicative constants and not by their order of growth.

Example 4: Counting binary digits

- Find the number of binary digits in the binary representation of a positive decimal integer.

ALGORITHM *Binary*(n)

//Input: A positive decimal integer n

//Output: The number of binary digits in n 's binary representation

$count \leftarrow 1$

while $n > 1$ **do**

$count \leftarrow count + 1$

$n \leftarrow \lfloor n/2 \rfloor$

return $count$

Example 4: Counting binary digits

- The time complexity is **proportional** to the **number of digits**.
- If there are **3,000 digits** in the number, the while loop **will iterate** 3,000 times.
- How many digits are there in n , asymptotically?
- When we divide n by 2, after k division we will get 1 as output.
- In other words, $\frac{n}{2^k}=1$
- Taking a logarithm, we have $k=\log_2(n)$
- Algorithm executes k times to find out the digit number.
- The above shows that the algorithm's time complexity is $\Theta(\log_2(n))$.

Summary

- Asymptotic Order of Growth (Big oh, Omega, and Theta)
- L'Hopital's rules, Stirling
- Nonrecursive Algorithms
 - Maximum Element
 - Element Uniqueness
 - Matrix multiplication
 - Counting Binary Digits

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition)* by Anany Levitin, Pearson.
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.